Homework 4

Gabriela Rubio

March 19, 2026

1. (a) **Algorithm Description:** For this algorithm we can do a traversal of the heap starting from the root, and initializing an empty set to track nodes to visit. For each node i we pop from the unexplored nodes set, if $A[i] \geq x$ we discard it (every descendant of i is also $\geq x$). If $A[i] < x$ then we increment a counter and add the children of i to the set of unexplored sets. If the counter reaches k, we return "yes," and if it empties before that, we return "no."

   (b) **Pseudocode:**

```
Algorithm: IsKSmallerThanX
Input: A minHeap A with n elements, a target number x,
and a target integer k
Output: Whether k is smaller is than x or not

count = 0
unexplored = {}

add 1 to unexplored

while unexplored is not empty:
    remove a node i from unexplored
    if A[i] >= x:
        continue
    else:
        count = count + 1
        if count == k:
            return "Yes"
        if i * 2 <= n:
            add (i * 2) to unexplored
        if  i * 2 +1 <= n
            add (i * 2 + 1) to unexplored

return "No"
```

   (c) **Correctness:** The algorithm is correct because it only counts the number of elements in the heap thar are less than x, and only returns "Yes" if and only if at least k elements exists. Pruning is valid because if a node is bigger than x, all of its children are also bigger than x, because in a min-heap the parent nodes are always smaller than the children. Every element that is smaller than x has all its ancestor be smaller than x as well, so we never prune this specific path.

   (d) **Time Analysis:** All the initializions are $O(1)$, as well as the comparisons. The loop itself is $O(k)$, because every node in unexplored is either greater than x and not explored every again, or is smaller than x and we check the children, and since we stop as soon as we get to k, we only iterate $1 + 2k$ times, which is $O(k)$.

2. (a) **Algorithm Description:** This algorithm performs a traversal in our BST. We store our best candidate so far in a variable. Given our number x, we compare it to our node, starting at the root. If the node is bigger, we save the node as a candidate, and check that the left child is better. If it's equal, we return x. If it's smaller, we check the right child.

(b) **Pseudocode:**

```
Algorithm: Successor(T, x)
Input: A balanced BST T and a number x
Output: The successor of x in T, or NULL

candidate = NULL
current = T.root
while current != NULL:
    if current.key == x:
        return x
    elif current.key > x:
        candidate = current
        current = current.left
    else:
        current = current.right
if candidate == NULL:
    return NULL
return candidate.key
```

(c) **Correctness:** The algorithm is correct because it splits up in three possible cases. If x is equal to a key in T, we correctly return x. In the case that x is greater than every key, we always go right and never find a candidate so we return NULL. If x has a successor but is not a key in T, we make sure that everytime we go left we save the best current candidate in case when we go left we find an unsuccessful candidate and we can just return, while going right doesn't change anything because it means that those keys are too small to be successors.

(d) **Time Analysis:** The algorithm performs $O(1)$ work per node, and goes from root-to-leaf. Since the path from root-to-leaf is $O(\log n)$, the total running time is also $O(\log n)$.

3. (a) **Design:** Augment T by adding a size property to each node. For each child that is added, add 1 to the size of each ancestor. Adding is still O(log n) because we only traverse to what we add, same as deleting. For checking the rank of a given number, we traverse the tree downwards and maintain a count. If we find a v.key = number, we return count + v.left.size + 1; if v.key ¡ number, we add v.left.size + 1 to the count and go right. If v.key ¿ number, go left without adding anything to count. When we reach the end of the tree, we return count + 1.

(b) **Algorithm:**

```
Algorithm: Rank(T, x)
Input: Augmented BST T and a number x
Output: rank(x) = 1 + (number of keys in T smaller than x)

count = 0
current = T.root
while current != NULL:
    if current.key == x:
        lsize = 0 if current.left == NULL else current.left.size
        return count + lsize + 1
    elif current.key < x:
        lsize = 0 if current.left == NULL else current.left.size
        count = count + lsize + 1
        current = current.right
    else:
        current = current.left
return count + 1
```

(c) **Main Theorem:** Search is unchanged in this algorithm, so still $O(\log n)$. For insert, we follow the regular BST path which is length $O(\log n)$ to the new node, and then increment

size for ancestor on the way back, which is $O(1)$ per node, and $O(\log n)$ total. Delete works the opposite way but symetrically: we traverse $O(\log n)$ nodes and decrement size for each ancestor on the path. Both operations are still $O(\log n)$.

4. (a) **Algorithm Description:** Find the LCA. From that node, we do an in-order traversal, only if we find v.key ¡ x_l we don't keep going to the left, and if v.key ¿ x_r we don't keep going to the right.

   (b) **Pseudocode:**

```
Algorithm RangeReport(T, xl, xr)
Input: BST T and range [xl, xr]
Output: All keys in the range [xl, xr] in sorted order

split = LCA(T, xl, xr)
InOrderTraversal(split, xl, xr)

Algorithm InOrderTraversal(v, xl, xr)
Input: Node to start traversing from, and rage [xl, xr]
Output: It traverses in-order from v,
and "reports/output/print" all the keys in [xl,xr]
    if v == NULL: return
    if v.key > xl:
        InOrderTraversal(v.left, xl, xr)
    if xl <= v.key <= xr:
        output v.key
    if v.key < xr:
        InOrderTraversal(v.right, xl, xr)
```

   (c) **Correctness:** We use the LCA algorithm taught in class, which we know is correct. Then, we just do an in-order traversal, because every key in the range is within the subtree rooted at the LCA, so we don't miss anything. We prune left if v.key $\leq x_l$ since no key in the left subtree can be $\geq x_l$, and prune right if v.key $\geq x_r$. so no key in range is skipped. Since we recurse left before outputting and right after, output is in sorted BST order.

   (d) **Time Analysis:** Finding the LCA takes $O(\log n)$. Visiting and outputting each of the k nodes in the range is $O(1)$. Nodes outside the range are only visited along the boundary paths toward x_l and x_r, and each takes $O(\log n)$. The total time then $O(k + \log n)$.

5. (a) **Design:** Augment T by adding a subtreeSum, which sums all keys in the subtree rooted at that node. When inserting or deleting, we update subtreeSum for all ancestors on the traversal path. Keep a sum variable and find the first node that is within range. We add the key of this node to the sum, and then traverse left towards v_l and right towards v_r. When traversing left, if v.key =¿ v_l, we add v.key + v.right.subtreeSum to our sum and traverse its left child; if it's smaller, traverse its right child. Traverse v_r symmetrically. We return our sum when we reach the end of the tree.

   (b) **Algorithm:**

```
Algorithm: RangeSum(T, xl, xr)
Input: Augmented BST T and a range [xl, xr]
Output: Sum of all keys in the range [xl, xr]

sum = 0
v = T.root
while v != NULL:
    if v.key < xl:
        v = v.right
    elif v.key > xr:
        v = v.left
    else:
        break
```

```
if v == NULL: return 0
sum = v.key

left = v.left
while left != NULL:
    if left.key >= xl:
        sum = sum + left.key
        if left.right != NULL:
            sum = sum + left.right.subtreeSum
        left = left.left
    else:
        left = left.right

right = v.right
while right != NULL:
    if right.key <= xr:
        sum = sum + right.key
        if right.left != NULL:
            sum = sum + right.left.subtreeSum
        right = right.right
    else:
        right = right.left

return sum
```

(c) **Main Theorem:** Search is unchanged in this algorithm, so still $O(\log n)$. For insert, we follow the regular BST path which is length $O(\log n)$ to the new node, and then update subtreeSum for ancestor on the way back by adding the new key, which is $O(1)$ per node, and $O(\log n)$ total. Delete works the opposite way but symetrically: we traverse $O(\log n)$ nodes and subtract the deleted key from subtreeSum for each ancestor on the path. Both operations are still $O(\log n)$.